Learned Search Policies for Proof Search

What Machine Learning Can and Cannot Buy an Automated Theorem Prover

Version 2

Brian Tenneson
M.S., Applied Data Science
Currently Unaffiliated
email: btenneson2301@baypath.edu

July 27, 2026

Version 2 — July 27, 2026
Supersedes *Learning Theorem Space Geometry* and the machine-learning sections of *Toward a Geometry of Theorem Space*

https://btenneson2301.substack.com

Abstract

Version 1 proposed that machine learning improves theorem proving because it approximates the geometry of theorem space, and stated three claims as theorems and four as conjectures. This version withdraws the geometric framing as the organizing hypothesis and replaces it with one that is already formalized in *Depths of a Simulation*: a learned proof-search heuristic is a *nondeterministic proof-search policy* in the sense of Part I, Section 5 of the merged edition. That identification is not decorative. It makes three things provable that were previously asserted.

First, a hard floor: no policy, however trained, produces a target formula earlier than its closure depth (Theorem 3.2). Learning can reduce the *breadth* of proof search and never its *depth*. Second, an exact account of what pruning costs: the Branch-Covering Theorem says proof-covering policies retain completeness and reach the proof horizon, while a two-line example shows that a greedy policy — one that commits to a single successor — can fail to reach provable targets at all (Example 3.8). Third, a correct replacement for the Geodesic Proof Conjecture, which as stated is a tautology: the substantive question is whether an *admissible* heuristic can be learned, since only admissibility preserves the optimality guarantee of the Shortest-Path ATP Principle, and learned heuristics are trained for accuracy rather than for under-estimation.

The three claims labelled theorems in Version 1 are empirical hypotheses about learned models, not consequences of any definition, and are restated as such. The architecture proposal is updated: the embed-and-navigate design reflects roughly the 2019–2021 state of the field, whereas systems that now lead benchmarks generate candidate proofs with a language model, verify them with a proof assistant, and train on verified traces. An evaluation section is added covering benchmark contamination, which is the dominant methodological problem in this area and was absent from Version 1.

Keywords: automated theorem proving, machine learning, proof search, search policy, premise selection, proof horizon, admissible heuristic, A* search, benchmark contamination, algorithm selection.

1 What this version corrects

Version 1 advanced the hypothesis that "machine learning succeeds because it approximates theorem-space geometry," developed a four-component architecture, and stated three theorems and four conjectures. The following table records what survives.

Version 1	**Status**
Theorem: Embedding Stability	Not a theorem. No regularity assumptions are specified, no convergence notion is defined, and no proof is given. It is an empirical claim about training runs; restated as Hypothesis 5.2.
Theorem: ATP Preference	Not a theorem. "Statistically distinguishable regions" is an empirical claim about solver behaviour; restated as Hypothesis 5.3, where it is also the best-supported of the three.
Theorem: Navigation Compression	Not a theorem as stated: the entropy is of an unspecified distribution. A correct and provable version, bounding what any policy can achieve, is Theorem 3.2 together with Proposition 3.4.
Conjecture: Geodesic Proof	Vacuous. Shortest proofs correspond to shortest paths by the construction of the theorem graph. Replaced by the admissible-heuristic question of Section 4.
Conjecture: Theorem-Space Geometry	Retained as Hypothesis 5.1, narrowed to the claim actually supported by evidence: local similarity is learnable and useful for premise selection.
Conjecture: ATP Specialization	Retained; this is the portfolio-selection problem, which has substantial prior art that Version 1 did not cite.
Conjecture: Universal Navigation	Retained as Hypothesis 5.4, with the contamination caveat of Section 7, without which it cannot be tested.
$T(F)$ = "the inference-closed consequences of F"	Ill-formed: consequences are relative to a hypothesis set. Corrected in Definition 2.1.

The deeper problem was structural. Version 1 invoked the SIC framework in a section heading and then made no use of it: delete every occurrence of the word and the document is unchanged. This version uses the apparatus, and the apparatus turns out to answer the paper's central question.

2 The identification that does the work

Definition 2.1 (Theorem graph, corrected)**.** Let $F = (x, y, z)$ be a formal system and let $\Gamma \subseteq y$ be a hypothesis set. The consequence set is $\mathrm{Con}_F(\Gamma)$; there is no consequence set of F alone. The *state theorem graph* for Γ has as vertices the proof states $p \subseteq y$ with $\Gamma \subseteq p$, and a directed edge $p \to p \cup \{r(a_1, \ldots, a_k)\}$ labelled $(r, a_1, \ldots, a_k)$ whenever $r \in z_k(F)$, each $a_i \in p$, and $r(a_1, \ldots, a_k)$ is defined. Restricting to edges that add a new formula yields a directed acyclic graph.

Definition 2.2 (Search policy)**.** An *admissible one-step extension* at $p \subseteq Y$ is a finite set $\Delta \subseteq Y$ each of whose members is a direct consequence of p. A *nondeterministic proof-search policy* on Y is a function Σ assigning to each pair (p, m) a nonempty set $\Sigma(p, m)$ of admissible one-step extensions at p. A *branch* from Γ is a sequence $\langle p_1, p_2, \ldots \rangle$ with $p_1 = \Gamma$ and $p_{m+1} = p_m \cup \Delta_m$ for some $\Delta_m \in \Sigma(p_m, m)$.

Remark 2.3 (A trained model is a policy). This is the whole point of the present version. A neural premise selector, a tactic-prediction model, a language model emitting proof steps, and a learned value function guiding best-first search are all, in the vocabulary of Definition 2.2, functions from a proof state to a set of candidate one-step extensions. The learned parameters determine *which* extensions are offered and in what order; they do not change what an extension is. A learned prover is therefore a pair (policy, execution strategy), and every theorem below about policies applies to it without modification, whatever its architecture and however it was trained.

The consequence is that the questions Version 1 posed as open — how much can learning help, and what does it risk — are partly settled already, by results proved for arbitrary policies.

3 What is provable

Definition 3.1 (Closure depth)**.** Let $D_F(A) := A \cup \{\beta : \beta \text{ is a direct consequence of } A\}$ and let $D_F^0(\Gamma) := \Gamma$, $D_F^{m+1}(\Gamma) := D_F(D_F^m(\Gamma))$. The *closure depth* of φ from Γ is

$$\delta_F(\Gamma, \varphi) := \min\{m \geq 0 : \varphi \in D_F^m(\Gamma)\},$$

with $\delta_F(\Gamma, \varphi) := \infty$ if no such m exists.

Theorem 3.2 (Depth floor for every policy)**.** *Let Σ be any nondeterministic proof-search policy on Y, let $\Gamma \subseteq Y$, and let $b = \langle p_1, p_2, \dots \rangle$ be any branch of the search generated by Σ from Γ. Then for every $m \in \mathbf{N}^+$,*

$$p_m \subseteq D_F^{m-1}(\Gamma).$$

Consequently, if $\varphi \in p_m$ then $m \geq \delta_F(\Gamma, \varphi) + 1$: no policy produces φ at a stage earlier than its closure depth.

Proof. Induction on m. For $m = 1$, $p_1 = \Gamma = D_F^0(\Gamma)$. Suppose $p_m \subseteq D_F^{m-1}(\Gamma)$. Every $\beta \in \Delta_m$ is a direct consequence of p_m, hence a direct consequence of a subset of $D_F^{m-1}(\Gamma)$, hence $\beta \in D_F(D_F^{m-1}(\Gamma)) = D_F^m(\Gamma)$. Since also $p_m \subseteq D_F^{m-1}(\Gamma) \subseteq D_F^m(\Gamma)$, we get $p_{m+1} = p_m \cup \Delta_m \subseteq D_F^m(\Gamma)$.

For the consequence: if $\varphi \in p_m \subseteq D_F^{m-1}(\Gamma)$ then $\delta_F(\Gamma, \varphi) \leq m - 1$. □

Remark 3.3 (Learning reduces breadth, not depth). Theorem 3.2 is the precise form of what Version 1 called navigation compression, and it points the opposite way from the informal statement. Training cannot move a target closer than its closure depth, because the bound holds for *every* policy and is proved from the definition of an admissible extension alone. No architecture, no amount of data, and no reinforcement signal affects it. What a policy can do is control $|p_m|$: how much of $D_F^{m-1}(\Gamma)$ it actually materializes on the way. Unguided closure materializes all of it; a good policy materializes a thin subset containing the target.

Proposition 3.4 (The quantity learning acts on)**.** *For a branch b generated by Σ and any m,*

$$\Gamma \subseteq p_m \subseteq D_F^{m-1}(\Gamma),$$

and both inclusions can be strict. The unguided canonical rule-enumeration machine realizes the upper bound at every stage; it arises as a branch of a policy exactly when F is finitely branching *on Y, meaning $D_F(p) \setminus p$ is finite for every $p \subseteq Y$, since Definition 2.2 requires one-step extensions to be finite. For infinitely branching systems the upper bound is still an upper bound, but it is not attained by any single branch. The improvement available to a learned policy is therefore exactly the ratio $|p_m|/|D_F^{m-1}(\Gamma)|$, and any experimental claim of speedup should report it.*

Proof. The inclusions are Theorem 3.2 together with $p_1 = \Gamma$ and monotonicity of the branch. The canonical machine has $C(\Gamma, m) = D_F^{m-1}(\Gamma)$ by definition, giving equality on the right. Strictness on the left is immediate for any policy that ever fires a rule. □

Proposition 3.5 (Closure depth is below proof length)**.** *For every $\Gamma \subseteq y$ and $\varphi \in y$ with $\Gamma \vdash_F \varphi$,*

$$\delta_F(\Gamma, \varphi) + 1 \;\leq\; \|\varphi\|_F(\Gamma).$$

Proof. Let $\pi = \langle \pi(1), \ldots, \pi(n) \rangle$ be a proof from Γ. We show by induction on j that $\pi(j) \in D_F^{j-1}(\Gamma)$. For $j = 1$, the only admissible first line is a hypothesis, since a rule needs at least one premise, so $\pi(1) \in \Gamma = D_F^0(\Gamma)$. For the step, $\pi(j)$ is either in $\Gamma \subseteq D_F^{j-1}(\Gamma)$ or a direct consequence of $\{\pi(1), \ldots, \pi(j-1)\} \subseteq D_F^{j-2}(\Gamma)$ by the inductive hypothesis and monotonicity of D_F, hence lies in $D_F^{j-1}(\Gamma)$. Taking $j = n$ and $\pi(n) = \varphi$ gives $\delta_F(\Gamma, \varphi) \leq n - 1$; minimizing over proofs gives the claim. □

Remark 3.6 (The two bounds are consistent, and the gap is the point). Theorem 3.2 places a floor at $\delta_F(\Gamma, \varphi) + 1$ and Theorem 3.7 places a ceiling at $\|\varphi\|_F(\Gamma)$ for proof-covering policies. Proposition 3.5 shows the floor lies below the ceiling, so the two are compatible; and the gap between them is exactly the room a policy has to work in. The gap is real and can be large: a proof whose lines could all have been derived in parallel has closure depth far below its linear length, because a linear proof must list its premises in sequence while D_F applies every available rule at once. A policy that exploits available parallelism approaches the floor; one that does not approaches the ceiling.

3.1 What pruning costs

Theorem 3.7 (Branch covering, restated)**.** *Call Σ* proof-covering *if for every Γ and every proof $\pi = \langle \pi(1), \ldots, \pi(n) \rangle$ from Γ there is a branch b with $\{\pi(1), \ldots, \pi(j)\} \subseteq p_j$ for all $j \leq n$. If Σ is proof-covering and $\Gamma \vdash_F \varphi$, then some branch reaches φ at a stage no later than $\|\varphi\|_F(\Gamma)$, the shortest proof length.*

Proof. Take a shortest proof π of φ, of length $n = \|\varphi\|_F(\Gamma)$. Proof-covering supplies a branch with $\{\pi(1), \ldots, \pi(j)\} \subseteq p_j$ for each j, so $\varphi = \pi(n) \in p_n$. □

Example 3.8 (A greedy policy loses completeness)**.** Let $y = \{a, b, c\}$ with two unary rules, $r(a) = b$ and $s(a) = c$, and let $\Gamma = \{a\}$, target c. Define $\Sigma(p, m) := \{\{b\}\}$ whenever $a \in p$, and $\Sigma(p, m) := \{\varnothing\}$ otherwise. This is a legitimate policy in the sense of Definition 2.2: $\{b\}$ is an admissible one-step extension at any state containing a. Its unique branch is $\{a\}, \{a, b\}, \{a, b\}, \ldots$, which never contains c, although $\Gamma \vdash_F c$ in one rule application.

Remark 3.9 (Why this is the central practical risk). Example 3.8 is the formal shadow of a system that commits to its top-ranked tactic and does not backtrack. The policy is not defective as a policy; it is simply not proof-covering, and Theorem 3.7 does not apply to it. A learned model ranks successors, and the ranking is only as reliable as its training distribution; on inputs unlike anything in the corpus the top-ranked successor can be systematically wrong. This is why deployed systems retain search — best-first with a queue, or sampling many candidate proofs — rather than following the model greedily. Completeness is a property of the search wrapper, not of the model.

The honest engineering statement is therefore: *the model supplies the ordering; the wrapper supplies the guarantee.* A research program on learned proof search should specify both, and Version 1 specified neither.

3.2 Lemma libraries have an exact theory already

Theorem 3.10 (Conservative compilation, restated)**.** *Let $\mathcal{T} = \{(\Gamma_i, \varphi_i) : 1 \le i \le N\}$ be finite with each $\Gamma_i \vdash_F \varphi_i$. Then there is a finite set D of derived rules such that the canonical target-search machine over $F^D = (x, y, z \cup D)$ reaches φ_i from Γ_i within two stages, for every i; and every theorem of F^D is a theorem of F.*

Remark 3.11 (Premise selection is compilation with a learned index)*.* Theorem 3.10 is the exact statement of what a lemma library buys, and it is already proved. A retrieval-based premise selector over Mathlib or the Archive of Formal Proofs is doing two things at once: the library supplies D, and the learned component supplies an *index* into D — a guess at which few of the hundreds of thousands of available lemmas are relevant to the current goal. The compilation theorem says the horizon collapses once the right derived rule is available; the learning problem is retrieval, not horizon reduction.

This is a more accurate description of why premise selection works than "approximating geometry," and it explains the observed shape of the results: gains are large on goals whose proofs are short given the right lemmas, and small on goals that need genuinely new intermediate structure — for which no D assembled from the existing library suffices.

4 The geodesic question, corrected

Remark 4.1 (Why the Geodesic Proof Conjecture is vacuous)*.* Version 1 conjectured that "shortest proofs correspond to geodesics under an appropriate theorem-space metric." In the state theorem graph of Definition 2.1 with unit edge costs, the graph distance from Γ to the nearest state containing φ *is* the least number of one-formula expansions needed, and a geodesic is by definition a path realizing the graph distance. The conjecture asserts that shortest paths are shortest paths. Nothing is claimed.

The substantive question in the vicinity is the one classical heuristic search already poses.

Definition 4.2 (Admissible learned heuristic)**.** Let $d^*(p, \varphi)$ denote the true graph distance from state p to the nearest state containing φ. A function h is *admissible* for φ if $h(p, \varphi) \le d^*(p, \varphi)$ for every p, and *consistent* if $h(p, \varphi) \le 1 + h(q, \varphi)$ for every edge $p \to q$.

Proposition 4.3 (What admissibility buys and costs)**.** *Assume the reachable portion of the state theorem graph is locally finite and effectively enumerable, as in the Shortest-Path ATP Principle of Part I. If h is admissible then A* search on that graph with cost $g(p) + h(p, \varphi)$ returns a proof of least length, and if h is consistent no state is expanded twice. If h is inadmissible — if it overestimates anywhere on an optimal path — the returned proof may be longer than the shortest, and the guarantee of the Shortest-Path ATP Principle is lost.*

Proof. This is the standard optimality theorem for A* [5, 6], applied to the graph of Definition 2.1, whose edges have unit cost; local finiteness is what makes the open list well defined at each step. The failure direction is the standard counterexample: if h overestimates at a node on the unique optimal path, that node may be dequeued after a suboptimal goal. □

Remark 4.4 (The actual research question)*.* Learned heuristics are trained to minimize a prediction error — typically squared error or a ranking loss against observed proof lengths. Nothing in that objective encourages under-estimation, and a model with small average error will overestimate on

roughly half its inputs. Learned heuristics are therefore essentially never admissible, and guided search using them is a heuristic procedure without an optimality guarantee.

Two well-posed questions follow, and they are the ones that should replace the Geodesic Proof Conjecture.

1. Can an admissible heuristic for proof distance be learned — for instance by training against a lower bound on remaining distance rather than the observed distance, or by shifting a trained model down by a certified error bound?
2. If not, what is the cost? For ε-admissible heuristics, with $h \leq (1+\varepsilon)d^*$, A* returns a proof within a factor $1+\varepsilon$ of shortest. Can a trained model be certified ε-admissible on a distribution, and what ε is achievable?

Both are answerable, both have classical theory behind them, and neither requires the geometry hypothesis.

5 Hypotheses, stated as hypotheses

Hypothesis 5.1 (Local structure is learnable)**.** There is a learnable embedding of formulas under which co-occurrence in proofs is predictable, in the sense that a retrieval model trained on a proof corpus selects relevant premises with precision substantially above a frequency baseline.

This is the defensible residue of the Theorem-Space Geometry Conjecture. Note what it does *not* say: nothing about a global metric, nothing about geodesics, nothing about convergence to a canonical structure. It says local relevance is predictable, which is what premise-selection results support.

Hypothesis 5.2 (Embedding stability)**.** Embeddings trained on distinct proof corpora agree, up to an affine transformation, on the relative similarity of formulas occurring in both.

Stated this way the claim is falsifiable: fix a shared subset of formulas, train on disjoint corpora, and measure rank correlation of pairwise similarities. Version 1 asserted convergence "under suitable regularity assumptions" without saying what converges, in what topology, or to what.

Hypothesis 5.3 (Solver specialization)**.** Distinct ATP systems have distinguishable competence profiles: there is a learnable map from goal features to the solver most likely to succeed, and a portfolio using it outperforms any single constituent solver.

This is the best-supported of the three, and it is not new. It is the *algorithm selection* problem, with a substantial literature: portfolio solvers have been standard in satisfiability competitions for two decades, and Isabelle's Sledgehammer has dispatched goals to a portfolio of external provers since well before the neural era. Version 1 proposed it as a novel theorem without citing the field. The research contribution available here is not the existence of specialization but its *structure*: whether solver competence is predictable from cheap syntactic features, and whether the partition is stable across libraries.

Hypothesis 5.4 (Cross-library transfer)**.** A policy trained on one formal library retains a substantial fraction of its advantage when evaluated on a different library with a different logical foundation.

Untestable without the controls of Section 7.

6 Architecture, updated

The four-component design of Version 1 — embed, estimate distance, select a solver, navigate — describes the neural-guidance architectures of roughly 2019–2021. It is not what leads benchmarks now. The prevailing design is:

1. a language model proposes a candidate proof, or a next tactic, in the concrete syntax of a proof assistant;
2. the proof assistant *verifies* it, giving a ground-truth reward with no learned critic and no possibility of a hallucinated proof being accepted;
3. successful traces are added to the training set and the model is retrained or updated by reinforcement learning;
4. search — best-first over tactic candidates, or sampling many whole proofs — wraps the model to restore the completeness that Example 3.8 shows a greedy policy lacks.

The verifier is the essential component and the one Version 1 omitted. It is what makes the training signal trustworthy and what makes the whole enterprise safe: an unsound proof is rejected mechanically, so the learned component can be arbitrarily unreliable without compromising correctness. In the vocabulary of Part I, the model supplies Σ and the assistant supplies the guarantee that every accepted branch is rule-driven.

Remark 6.1 (Retain the solver selector). Of the four Version 1 components the ATP selector is worth keeping as stated: it is a well-defined supervised problem, has an established literature, and yields measurable gains without requiring any of the geometric apparatus. It is the component to build first.

7 Evaluation and contamination

Version 1's experimental program has five phases and no evaluation methodology. This is the most serious practical omission, because the dominant failure mode in this field is measuring memorization.

Remark 7.1 (The contamination problem). Models are trained on corpora that overlap the benchmarks. A system trained on Mathlib and evaluated on Mathlib-derived problems may be retrieving rather than proving, and the resulting number is uninterpretable. The field has responded with perturbation benchmarks, on which every evaluated model loses accuracy when problem statements are altered in meaning-preserving ways, and with post-cutoff evaluation sets drawn from competitions held after the training cutoff.

Remark 7.2 (Benchmarks, and which are still informative). MiniF2F, 488 competition problems formalized in several systems, is close to saturated: leading provers now exceed ninety percent on its test split, so differences between strong systems are within noise and largely reflect contamination. PutnamBench, 1692 formalizations of 640 Putnam problems across Lean, Coq and Isabelle, remains discriminating: reported results on its Lean subset of 672 problems are on the order of 90 solved, well under fifteen percent. Figures across the three target languages are not comparable, so the subset should always be named when a number is quoted. Benchmark choice should follow headroom, and a saturated benchmark should be reported only for continuity with prior work.

A defensible protocol:

1. *Temporal split.* Train only on library commits predating a fixed date; evaluate on theorems added after it. This is the only split that approximates the deployment condition.

2. *Perturbation control.* Report accuracy on meaning-preserving rewrites of the evaluation statements alongside the originals. A large gap indicates memorization.
3. *Compute-matched baselines.* Report the unguided baseline at equal wall-clock and equal node expansions. A learned policy that wins on wall clock but loses on expansions has bought its gain with hardware.
4. *Report the breadth ratio.* By Proposition 3.4 the quantity a policy can improve is $|p_m|/|D_F^{m-1}(\Gamma)|$. Reporting it separates genuine search reduction from faster iteration.

8 Falsifiable predictions

The following are consequences of the position taken here, and each would count against it if it failed.

Prediction 8.1. No learned system will exhibit a proof found at a stage below its closure depth, for any goal, in any architecture. This follows from Theorem 3.2 and would, if violated, indicate an implementation error rather than a discovery — most likely a hidden lemma library, which changes F and hence changes D_F.

Prediction 8.2. Systems that follow a learned policy greedily will exhibit systematic incompleteness on goals whose shortest proof begins with a low-ranked step, and restoring search will recover them. This is Example 3.8 at scale.

Prediction 8.3. Gains from premise selection will correlate with the availability of a sufficient lemma set in the library, and will fall sharply on goals requiring intermediate structure absent from it — because by Theorem 3.10 the compilation effect is bounded by what D contains.

Prediction 8.4. Off-the-shelf learned distance estimators will prove inadmissible on a measurable fraction of states, and A* using them will return non-shortest proofs at a rate increasing with that fraction.

9 Revised experimental program

Phase 0 (new). Establish the evaluation harness before any model is trained: temporal split, perturbation set, compute-matched unguided baseline, and instrumentation for the breadth ratio of Proposition 3.4. Nothing measured before this exists is interpretable.
Phase I. Build the solver selector of Hypothesis 5.3. Smallest scope, established prior art, immediate measurable payoff, and independent of every contested claim in this paper.
Phase II. Premise selection, evaluated as retrieval (precision at k against the premises actually used) rather than end-to-end, so that the retrieval quality is separated from the downstream prover.
Phase III. Test Hypothesis 5.2 directly: disjoint corpora, shared formula subset, rank correlation of similarities.
Phase IV. Investigate admissibility (Remark 4.4): can a distance estimator be trained or shifted to be admissible, and at what cost in tightness?
Phase V. Cross-library transfer (Hypothesis 5.4), which is only meaningful once Phases 0–III have established that the within-library effect is real.

10 Conclusion

The claim that machine learning helps proof search is correct and needs no defence. The claim that it helps *because it approximates theorem-space geometry* was doing no work: it was not used to derive anything, it was not formulated precisely enough to test, and the one conjecture it generated was a tautology.

Replacing it with the identification of Remark 2.3 — a trained model is a search policy — costs nothing and yields a hard limit (Theorem 3.2: learning reduces breadth, never depth), an exact account of what pruning risks (Example 3.8), a correct theory of what lemma libraries buy (Theorem 3.10), and a well-posed replacement for the geodesic question (Section 4). These are modest results. They have the advantage of being true, and of being consequences of a framework already built rather than of a metaphor imported for the occasion.

References

[1] Tenneson, B. (2026). *Depths of a Simulation and Formalized Self-Awareness*, Merged Edition. Part I, Sections 2, 5, 6 and 7.

[2] Li, Z., Sun, J., Murphy, L., Su, Q., Li, Z., Zhang, X., Yang, K., & Si, X. (2024). A survey on deep learning for theorem proving. *Conference on Language Modeling*.

[3] Tsoukalas, G., et al. (2024). PutnamBench: Evaluating neural theorem-provers on the Putnam Mathematical Competition. *NeurIPS Datasets and Benchmarks*.

[4] Zheng, K., Han, J. M., & Polu, S. (2022). MiniF2F: A cross-system benchmark for formal Olympiad-level mathematics. *ICLR*.

[5] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

[6] Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley.

[7] Rice, J. R. (1976). The algorithm selection problem. *Advances in Computers*, 15, 65–118.

[8] Xu, L., Hutter, F., Hoos, H. H., & Leyton-Brown, K. (2008). SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32, 565–606.

[9] Paulson, L. C., & Blanchette, J. C. (2010). Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. *IWIL*.

[10] The mathlib Community (2020). The Lean mathematical library. *CPP*.

[11] Isabelle Archive of Formal Proofs. https://www.isa-afp.org/

